

```

64 var x = o.left;
65 var y = o.top;
66
67 var ax = settings.accX;
68 var ay = settings.accY;
69 var th = t.height();
70 var wh = w.height();
71 var tw = t.width();
72 var ww = w.width();
73
74 if (y + th + ay >= b &&
75     y <= b + wh + ay &&
76     x + tw + ax >= a &&
77     x <= a + ww + ax) {
78     //trigger the custom event
79     if (!t.appeared) t.trigger('appear', settings.data);
80
81     } else {
82     //it scrolled out of view
83     t.appeared = false;
84     }
85
86 };
87
88 //create a modified fn with some additional logic
89 var modifiedFn = function() {
90
91     //mark the element as visible
92     t.appeared = true;
93
94     //is this supposed to happen only once?
95     if (settings.one) {
96
97         //remove the check
98         w.unbind('scroll', check);
99         var i = $.inArray(check, $.fn.appear.checks);
100         if (i >= 0) $.fn.appear.checks.splice(i, 1);
101     }
102
103     //trigger the original fn
104     fn.apply(this, arguments);
105
106 };
107
108 //bind the modified fn to the element
109 $.fn.appear.one('appear', settings.data, modifiedFn);
110
111 //bind the modified fn to the element
112 $.fn.appear(settings.data, modifiedFn);

```



MATERIA:

Programación Orientada a Objetos

Unidad 2 Principios de la Programación Orientada a Objetos.

Docente:

Grupo:

Unidad #2 - Principios de la Programación Orientada a Objetos.

2.1. Herencia

- 2.1.1. Clases padre e hija.
- 2.1.2. Uso de super y ejecución de constructores.
- 2.1.3. Métodos protegidos y sobrescritos (@Override).

2.1.4. La clase Object y sus métodos (toString, equals, etc).

2.2. Métodos y atributos protegidos.

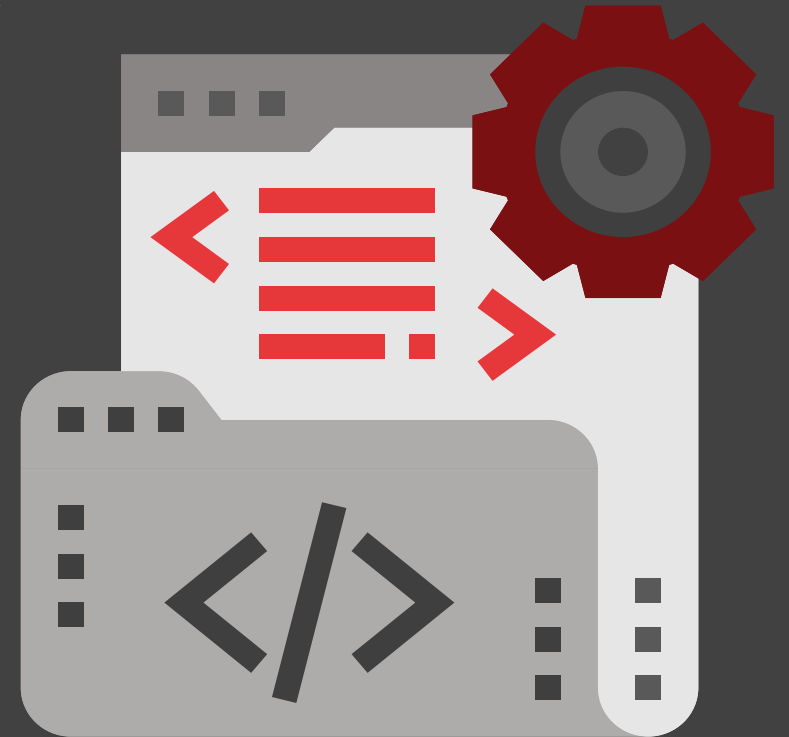
2.3. Clases finales.

2.1. Herencia

En programación orientada a objetos, la herencia es un mecanismo que permite que una clase nueva (llamada hija o subclase) reutilice, modifique o amplíe las características de otra clase existente (llamada padre, superclase o base)

Que es herencia

- Mecanismo de la POO que permite que una clase hija herede de una clase padre.
- La clase hija puede **reutilizar, modificar o ampliar** lo que define la clase padre.
- Relación: “**es un**” → Ejemplo: *Estudiante es una Persona*.



Ventajas de Herencia

- Reutilización de código.
- Claridad en jerarquías.
- Polimorfismo (una hija puede comportarse distinto, pero usarse como el padre).

Ejemplo de Herencia

```
class Persona { String nombre; int edad; } 1 usage 1 inheritor  
class Estudiante extends Persona { String carnet; } no usages
```

- 2.1.1. Clases padre e hija.

Clase padre: define atributos y métodos comunes.



Clase hija: hereda del padre con extends.



Puede agregar o especializar comportamientos.



Ejemplo Clase padre e hija

```
2 // Clase padre
3 class Persona { 1 usage 1 inheritor
4     String nombre; no usages
5     int edad; no usages
6 }
```

```
// Clase hija
class Estudiante extends Persona { no usages
    String carnet; no usages
}
```

Se define una **clase padre** llamada **Persona** con dos atributos: **nombre** y **edad**.
Luego, se crea una **clase hija** llamada **Estudiante** que **hereda de Persona** usando la palabra clave **extends**.
Estudiante obtiene automáticamente los atributos **nombre** y **edad** de **Persona**, y además añade un atributo propio: **carnet**.

2.1.2 Uso de super y ejecución de constructores

super se usa para:

Invocar el constructor de la clase padre.

Acceder a métodos o atributos del padre cuando fueron sobrescritos u ocultos

En un constructor, la llamada a `super(...)` debe ser la primera instrucción.

Si no escribes nada, Java inserta automáticamente `super()`

```
1 class Animal { 1 usage 1 inheritor
2     void sonido() { System.out.println("Sonido genérico"); } no usages
3 }
4
5 class Perro extends Animal { 1 usage
6     @Override no usages
7     void sonido() {
8         super.sonido(); // Llama al método del padre
9         System.out.println("Ladrido");
10    }
11 }
12
13 public class Main {
14     public static void main(String[] args) {
15         new Perro().sonido();
16     }
17 }
18
```

2.1.3. Métodos protegidos y sobrescritos (@Override).

Método sobrescrito (@Override)

```
✓ class Animal { 2 usages 1 inheritor
    protected void sonido() { System.out.println("Sonido genérico"); }
}

✓ class Perro extends Animal { 1 usage
    @Override 1 usage
    protected void sonido() {
        System.out.println("Ladrado");
    }
}

✓ public class Main {
    public static void main(String[] args) {
        Animal a = new Perro();
        a.sonido(); // Ladrado
    }
}
```

Reglas para sobrescribir

- La firma debe ser idéntica (nombre + parámetros).
- Tipo de retorno: igual o subtipo.
- Acceso: igual o más permisivo (ej: de `protected` a `public`).
- No se pueden sobrescribir métodos `final`, `static` o `private`.

Una **subclase puede redefinir** un método del padre.

Esto se llama **sobrescritura (override)**.

Se usa la anotación **@Override** para:

- Avisar al compilador.
- Evitar errores de firma.

2.1.4. La clase Object y sus métodos (toString, equals, etc).

```
class Persona { 2 usages
    String nombre; no usages
    Persona(String n) { this.nombre = n; } no usages

    @Override
    public String toString() {
        return "Persona: " + nombre;
    }
}

public class Main {
    public static void main(String[] args) {
        Persona p = new Persona( n: "Ana");
        System.out.println(p);    // Persona: Ana
    }
}
```

toString() → representación en texto.

equals(Object o) → compara objetos.

hashCode() → devuelve un número asociado al objeto (importante en colecciones).

```
class Usuario { 7 usages
    String id; no usages
    Usuario(String id) { this.id = id; } no usages

    @Override
    public boolean equals(Object o) {
        if (o instanceof Usuario) {
            Usuario u = (Usuario) o;
            return this.id.equals(u.id);
        }
        return false;
    }

    @Override
    public int hashCode() {
        return id.hashCode();
    }
}

public class Main {
    public static void main(String[] args) {
        Usuario u1 = new Usuario( id: "123");
        Usuario u2 = new Usuario( id: "123");

        System.out.println(u1.equals(u2));    // true
    }
}
```

2.2 Métodos y atributos protegidos

```
class Persona { 1 usage 1 inheritor
    protected String nombre; 1 usage
}

class Estudiante extends Persona { no usages
    void mostrar() { no usages
        System.out.println("Nombre: " + nombre);
    }
}
```



Atributo protegido

`nombre` es accesible en la subclase, pero no desde una clase externa.

```
class Padre { 1 usage 1 inheritor
    protected void saludar() { no usages 1 override
        System.out.println("Hola desde Padre");
    }
}

class Hijo extends Padre { no usages
    @Override no usages
    protected void saludar() {
        System.out.println("Hola desde Hijo");
    }
}
```



Métodos protegidos

Se usan igual que los atributos. Pueden ser redefinidos (override) en las hijas.

2.3. Clases finales.

Una **clase final** no puede ser heredada.
Se declara con la palabra clave **final**.
Ejemplo: **String** y **Math** en Java son clases **final**.

¿Por qué existen?

- Seguridad: evitar cambios en lógica base.
- Inmutabilidad: garantizar objetos que no se alteran.
- Diseño intencional: indicar que no debe extenderse.

```
final class Constantes { no usages
    public static final double PI = 3.1416; no usages
}
```

Nadie puede hacer class Otra **extends** Constantes (error de compilación).

```
final class CuentaBancaria { no usages
    private double saldo = 0; 2 usages

    public void depositar(double m) { saldo += m; } no usages
    public double getSaldo() { return saldo; } no usages
}
```

Protege la lógica de la cuenta para que no sea alterada por herencia.